Computer Networks Interview Guide

For Backend SWE & SRE/DevOps Interviews — Basic to Advanced

September 2026

How to use this guide

This is an **interview handbook**, not a textbook: every question is answered the way you'd actually say it out loud, with the specific terms an interviewer listens for bolded, the traps candidates fall into named explicitly, and the follow-up an interviewer typically asks next. Work top to bottom if you have a week; if you have a day, read Section 12 (the URL walkthrough — it touches almost every other section), then the Troubleshooting Playbook (Section 13), then skim the Question Index below for anything unfamiliar. This guide was built to serve **both Backend SWE and SRE/DevOps** candidates equally: conceptual/protocol questions sit alongside packet-level debugging and Linux tooling throughout, rather than in two separate tracks.

Contents

1 Research Notes: Queries and Sources iii

2 Interview Question Index v

3 OSI & TCP/IP Models 1

4 IP Addressing, Subnetting & CIDR 3

5 Transport Layer: TCP vs. UDP 5

6 DNS 8

7 HTTP/HTTPS Evolution 11

8 TLS & Network Security 14

9 NAT & Firewalls 17

10 Load Balancing & Reverse Proxies 19

11 Routing & Switching 21

12 End-to-End Flow: What Happens When You Type a URL 23

13 Troubleshooting Playbook **24**

14 Tools Quick Reference **28**

Final Notes **30**

1 Research Notes: Queries and Sources

Web search was available in this environment and was used, per instructions, before any outline was drafted. The category list and question selection below were shaped by what these searches actually returned, not by a pre-made syllabus. Where this guide calls something "frequently asked," it appeared across three or more of the sources below.

Search queries used

- *most common computer networks interview questions SWE SRE 2026*
- *site:geeksforgeeks.org computer networks interview questions*
- *Glassdoor networking interview questions SRE DevOps candidate experience*
- *what happens when you type a URL in browser interview question*
- *TCP vs UDP interview question three-way handshake congestion control*
- *DNS troubleshooting interview question dig nslookup commands SRE*
- *tcpdump ss traceroute interview troubleshooting packet loss latency*
- *TLS handshake interview question certificate MITM explain*
- *load balancing interview questions reverse proxy L4 vs L7*
- *subnetting CIDR interview questions GATE computer networks common questions*
- *HTTP/1.1 vs HTTP/2 vs HTTP/3 QUIC interview questions differences*
- *"site is down" troubleshooting playbook network SRE incident interview scenario*
- *NAT firewall interview questions stateful vs stateless*
- *RFC 793 TCP RFC 2131 DHCP RFC 1035 DNS interview reference*

Sources consulted

- **Candidate-experience / prep aggregators:** Glassdoor interview-experience pages for SRE/DevOps roles at Canonical, TikTok, Nutanix, and Google (`glassdoor.com/Interview/...`); GeeksforGeeks CN interview tracks (`geeksforgeeks.org/computer-networks/...`); InterviewBit Networking Interview Questions (`interviewbit.com/networking-interview-questions`); Hello Interview's Networking Essentials for System Design (`hellointerview.com/learn/system-design/core-concepts/networking-essentials`).
- **Canonical/standards references:** RFC 793 (original TCP spec, obsoleted by RFC 9293) and RFC 9293 at `rfc-editor.org` / `datatracker.ietf.org`; RFC 1035 (DNS) at `datatracker.ietf.org/doc/rfc1035`; RFC 2131 (DHCP) at `datatracker.ietf.org/doc/rfc2131`; RFC 9438 (CUBIC congestion control).
- **Academic / exam-syllabus completeness check:** GATE CS Computer Networks previous-year questions and solutions via GeeksforGeeks GATE archive and GATE Vidyalay (`gatevidyalay.com`) — used specifically to confirm subnetting/CIDR numeric-question coverage is representative of what's actually asked.

- **Practical tooling / Linux docs:** community write-ups on `dig` / `nslookup` troubleshooting workflows, `tcpdump` / `ss` / `traceroute` / `mtr` packet-loss and latency diagnosis guides, and AWS re:Post's VPC-to-on-prem connectivity troubleshooting notes.
- **Real-world engineering explanations:** Cloudflare's HTTP/2 vs HTTP/1.1 performance write-up (`cloudflare.com/learning/performance/http2-vs-http1.1`); PubNub, AlgoMaster, and GeeksforGeeks explainers on HTTP/1.1 vs HTTP/2 vs HTTP/3 (QUIC); loadbalancer.org's L4/L7/GSLB comparison; Google SRE Book chapters on incident response and managing incidents (`sre.google/sre-book/managing-incidents`, `sre.google/workbook/incident-response`).
- **Security fundamentals:** SSLDragon and Medium write-ups on how the TLS handshake and certificate-chain verification specifically prevent MITM attacks; IPWithEase's stateful-vs-stateless firewall comparison.

A note on freshness

Search results were treated as directional (what topics and phrasings interviewers actually use) rather than as the literal source of every fact in this guide — protocol mechanics (TCP state machine, DNS record types, TLS 1.3 handshake steps) were cross-checked against the RFCs and standard references above rather than taken verbatim from any single blog post.

2 Interview Question Index

ID	Difficulty	Category	Section	Page
Q1	Basic	OSI & TCP/IP Models	OSI & TCP/IP Models	1
Q2	Intermediate	OSI & TCP/IP Models	OSI & TCP/IP Models	1
Q3	Advanced	OSI & TCP/IP Models	OSI & TCP/IP Models	2
Q4	Basic	IP Addressing & Subnetting	IP Addressing, Subnetting & CIDR	3
Q5	Intermediate	IP Addressing & Subnetting	IP Addressing, Subnetting & CIDR	3
Q6	Intermediate	IP Addressing & Subnetting	IP Addressing, Subnetting & CIDR	3
Q7	Advanced	IP Addressing & Subnetting	IP Addressing, Subnetting & CIDR	4
Q8	Basic	Transport Layer (TCP/UDP)	Transport Layer: TCP vs. UDP	5
Q9	Basic	Transport Layer (TCP/UDP)	Transport Layer: TCP vs. UDP	5
Q10	Intermediate	Transport Layer (TCP/UDP)	Transport Layer: TCP vs. UDP	6
Q11	Advanced	Transport Layer (TCP/UDP)	Transport Layer: TCP vs. UDP	6
Q12	Advanced	Transport Layer (TCP/UDP)	Transport Layer: TCP vs. UDP	7
Q13	Basic	DNS	DNS	8
Q14	Intermediate	DNS	DNS	8
Q15	Intermediate	DNS	DNS	9
Q16	Advanced	DNS	DNS	9
Q17	Basic	HTTP/HTTPS Evolution	HTTP/HTTPS Evolution	11
Q18	Intermediate	HTTP/HTTPS Evolution	HTTP/HTTPS Evolution	11
Q19	Advanced	HTTP/HTTPS Evolution	HTTP/HTTPS Evolution	12
Q20	Advanced	HTTP/HTTPS Evolution	HTTP/HTTPS Evolution	12
Q21	Basic	TLS & Network Security	TLS & Network Security	14
Q22	Intermediate	TLS & Network Security	TLS & Network Security	14
Q23	Advanced	TLS & Network Security	TLS & Network Security	15
Q24	Advanced	TLS & Network Security	TLS & Network Security	16
Q25	Basic	NAT & Firewalls	NAT & Firewalls	17
Q26	Intermediate	NAT & Firewalls	NAT & Firewalls	17
Q27	Advanced	NAT & Firewalls	NAT & Firewalls	18
Q28	Basic	Load Balancing & Reverse Proxies	Load Balancing & Reverse Proxies	19
Q29	Intermediate	Load Balancing & Reverse Proxies	Load Balancing & Reverse Proxies	19
Q30	Advanced	Load Balancing & Reverse Proxies	Load Balancing & Reverse Proxies	20
Q31	Basic	Routing & Switching	Routing & Switching	21

ID	Difficulty	Category	Section	Page
Q32	Intermediate	Routing & Switching	Routing & Switching	21
Q33	Advanced	Routing & Switching	Routing & Switching	22
Q34	Advanced (Synthesis)	End-to-End Flow	End-to-End Flow: What Happens When You Type a URL	23
T1–T7	Scenario	Troubleshooting Playbook	Troubleshooting Playbook	24

7. Application Layer
6. Presentation Layer
5. Session Layer
4. Transport Layer
3. Network Layer
2. Data Link Layer
1. Physical Layer

3 OSI & TCP/IP Models

Q1. Name the OSI model's seven layers and the TCP/IP model's four (or five). Why do interviewers still ask about a model most real stacks don't implement layer-for-layer?

Difficulty: Basic **Category:** OSI & TCP/IP Models

The OSI layers, bottom to top: **Physical** (bits on a wire/radio), **Data Link** (frames, MAC addresses, switches), **Network** (packets, IP addresses, routers), **Transport** (segments, TCP/UDP, ports), **Session** (connection setup/teardown, often folded into the layers above in practice), **Presentation** (encoding/encryption format, also often folded in), **Application** (HTTP, DNS, SSH). The TCP/IP (Internet) model collapses this to four practical layers: **Link**, **Internet** (IP), **Transport** (TCP/UDP), **Application** (everything above transport, including what OSI calls session/presentation/application).

Interviewers ask this not because production code cares about seven boxes, but because it's the shared vocabulary for every other question in this guide — "that's a Layer 4 vs Layer 7 load balancer" or "that failure is at Layer 2, not Layer 3" only means something if you can place a concept in the stack instantly.

Common Misconceptions / Traps

Reciting the seven layers from memory but being unable to say which layer a **MAC address** (Layer 2) versus an **IP address** (Layer 3) versus a **port number** (Layer 4) belongs to. The list itself is worth almost nothing without being able to place real, concrete things on it.

Follow-Up Questions & What's Being Tested

Follow-up: "Where does TLS sit in this model?" Tests whether you know TLS doesn't map cleanly to either model — it typically runs above transport but below application (sometimes called Layer 5.5 informally), which is itself a useful thing to be able to say.

Q2. A switch and a router both "forward" traffic. What's actually different about how each makes that decision?

Difficulty: Intermediate **Category:** OSI & TCP/IP Models

A **switch** operates at Layer 2: it forwards **frames** based on destination **MAC address**, learned by watching source MAC addresses on each port and building a MAC address table (CAM table). It doesn't understand IP addresses at all in its core forwarding decision. A **router** operates at Layer 3: it forwards **packets** based on destination **IP address**, consulting a routing table to decide the next hop, and it rewrites the Layer 2 frame header at every hop (source/destination MAC change hop-to-hop; source/destination IP do not, barring NAT).

Common Misconceptions / Traps

Saying a router "changes the IP address" of a packet as it forwards it — it does not (unless NAT is involved). What changes hop-to-hop is the **Layer 2 framing** (MAC addresses); the IP packet's source/destination IP addresses stay the same end-to-end.

Follow-Up Questions & What's Being Tested

Follow-up: "What is a Layer 3 switch, then?" Tests whether you understand it's a switch that *also* does IP routing between VLANs in hardware — the naming reflects that modern switching/routing hardware has converged, not that the OSI layers themselves changed meaning.

Q3. Encapsulation: trace exactly what headers get added and stripped as an HTTP request travels from a browser, through a switch and a router, to a server.

Difficulty: Advanced **Category:** OSI & TCP/IP Models

At the sending host: the **application layer** produces an HTTP request (bytes). The **transport layer** wraps it in a TCP segment (adds source/destination **port**, sequence numbers). The **network layer** wraps that in an IP packet (adds source/destination **IP address**, TTL). The **link layer** wraps that in an Ethernet frame (adds source/destination **MAC address**) before putting bits on the wire.

At the first-hop **switch**: only the Ethernet frame is read (Layer 2) — the switch does not unwrap the IP packet at all, it just forwards based on destination MAC. At the **router**: the Ethernet frame *is* stripped (de-encapsulated) to read the IP packet's destination IP, the routing table is consulted, and a *new* Ethernet frame is built (new source/destination MAC for the next hop) around the *same* IP packet before re-transmitting. This repeats at every router hop. At the destination server, each layer's header is stripped in reverse order (link, then network, then transport) until the original HTTP bytes are handed to the application.

Common Misconceptions / Traps

Believing the IP packet itself is rebuilt at every router hop — it is not. Only the Layer 2 (Ethernet) framing around it is stripped and rebuilt at each hop; the IP header (and everything inside it) rides along unchanged from source to destination, aside from TTL decrementing and, if present, NAT rewriting the IP/port.

Follow-Up Questions & What's Being Tested

Follow-up: "Where does TTL get decremented, and why does it matter?" Tests whether you can connect encapsulation mechanics to a real, observable tool (traceroute, Section 13, works *because* routers decrement TTL and return an ICMP Time Exceeded message at TTL=0).

4 IP Addressing, Subnetting & CIDR

Q4. What is a subnet mask, and what does CIDR notation like /24 actually mean?

Difficulty: Basic **Category:** IP Addressing & Subnetting

A **subnet mask** splits an IP address into a **network portion** and a **host portion** — it's a bitmask the same length as the address (32 bits for IPv4) where 1-bits mark the network portion. **CIDR** (Classless Inter-Domain Routing) notation writes that mask as a single number: /24 means the first 24 bits are the network portion (mask 255.255.255.0), leaving 8 bits (256 addresses, 254 usable after reserving the network and broadcast addresses) for hosts.

Common Misconceptions / Traps

Forgetting that a /N network has 2^{32-N} total addresses but only $2^{32-N} - 2$ *usable host* addresses (the all-zeros host portion is the network address; all-ones is the broadcast address). This off-by-two error is the single most common subnetting mistake in interviews.

Follow-Up Questions & What's Being Tested

Follow-up: "What's the difference between classful and classless addressing?" Tests historical awareness — classful (A/B/C, fixed boundaries) wasted address space badly, which is exactly the scaling problem CIDR was invented to solve.

Q5. You need to carve 10.0.0.0/24 into 4 equal subnets. What are they?

Difficulty: Intermediate **Category:** IP Addressing & Subnetting

Going from /24 to 4 subnets requires borrowing 2 more bits from the host portion ($2^2 = 4$), giving four /26 networks, each with $2^{32-26} = 64$ addresses (62 usable): 10.0.0.0/26 (.0–.63), 10.0.0.64/26 (.64–.127), 10.0.0.128/26 (.128–.191), 10.0.0.192/26 (.192–.255). The general rule: to get N equal subnets, borrow $\lceil \log_2 N \rceil$ bits and add that to the original prefix length.

Common Misconceptions / Traps

Doing the bit math correctly but then reporting the wrong **broadcast address** for each subnet (it's the last address in the range, not the first address of the *next* subnet minus one arithmetic mistake away — though those are the same number, candidates often miscount).

Follow-Up Questions & What's Being Tested

Follow-up: "Now do the reverse: given 10.0.0.140, which of those four subnets is it in?" Tests whether you can go both directions (design → lookup) fluently, which is what real IP-plan review actually requires.

Q6. What are RFC 1918 private address ranges, and why do we still need them alongside NAT if IPv6 exists?

Difficulty: Intermediate **Category:** IP Addressing & Subnetting

RFC 1918 reserves three IPv4 ranges for private, non-internet-routable use: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. They exist because IPv4's 4.3 billion address space was exhausted years ago relative to the number of devices needing addresses, so private networks (homes, offices,

VPCs) reuse these ranges internally and rely on **NAT** (Section 9) to share a small number of public IPs. IPv6's 128-bit address space is large enough that this scarcity problem doesn't exist in principle, but IPv4/RFC 1918/NAT remain in near-universal production use because IPv6 adoption, while growing, is still incomplete, and most private cloud networking (VPC design, security-group defaults) is still IPv4-first by convention and tooling inertia.

Common Misconceptions / Traps

Claiming IPv6 "replaced" IPv4 in production — most real infrastructure today is IPv4-primary with IPv6 as a parallel/secondary path, not a full cutover, and interviewers with real infra experience will probe this if you overclaim IPv6 adoption.

Follow-Up Questions & What's Being Tested

Follow-up: "What happens if two companies with overlapping RFC 1918 ranges need to peer their VPCs?" Tests awareness of a genuine, common real-world pain point (address collision on VPC peering/merger), which has no clean fix short of re-addressing or NAT gateways between the two networks.

Q7. A router's table has entries for 10.0.0.0/8, 10.1.0.0/16, and 10.1.2.0/24. A packet arrives for 10.1.2.5. Which entry wins, and why?

Difficulty: Advanced **Category:** IP Addressing & Subnetting

10.1.2.0/24 wins. Routing table lookups use **longest prefix match**: when multiple entries match a destination address, the router selects the entry with the most specific (longest) prefix, not the first match or any priority ordering. All three entries technically "match" 10.1.2.5, but /24 is more specific than /16, which is more specific than /8.

Common Misconceptions / Traps

Assuming routing tables are checked top-to-bottom like firewall rules (first match wins). They are not — longest-prefix-match is a distinct algorithm, and getting this backwards is a common and serious misunderstanding for anyone who's worked mostly with firewall/ACL rule ordering.

Follow-Up Questions & What's Being Tested

Follow-up: "Why does CIDR aggregation (supernetting) matter for router performance?" Tests whether you understand that fewer, well-aggregated routing table entries mean faster longest-prefix-match lookups and smaller tables to propagate via BGP across the internet backbone.

5 Transport Layer: TCP vs. UDP

Q8. TCP vs. UDP — what's actually different, and how do you decide which to use?

Difficulty: Basic **Category:** Transport Layer (TCP/UDP)

TCP is connection-oriented: it establishes a session (the **three-way handshake**, below), guarantees ordered, reliable delivery via acknowledgments and retransmission, and provides **flow control** and **congestion control**. **UDP** is connectionless: it sends datagrams immediately with no handshake, no delivery guarantee, no ordering guarantee, and no built-in congestion control — any of that must be built by the application if it's needed. Use TCP when correctness/completeness of every byte matters more than latency (HTTP APIs, file transfer, database connections). Use UDP when low latency and tolerance for loss matter more than perfect delivery (DNS queries, video/voice calls, gaming, and — notably — QUIC/HTTP/3, which rebuilds reliability *on top of* UDP in user space rather than using TCP; Section 7).

Common Misconceptions / Traps

Saying "UDP is unreliable so you'd never use it for anything important." DNS (Section 6), most VoIP/video conferencing, and HTTP/3 all run over UDP in production at massive scale — "unreliable" means no delivery *guarantee*, not "unsuitable for serious use."

Follow-Up Questions & What's Being Tested

Follow-up: "Why did QUIC choose to build its own reliability on UDP instead of just using TCP?" Tests whether you understand **head-of-line blocking** (Q19) as the specific TCP limitation QUIC was designed to avoid.

Q9. Walk through the TCP three-way handshake.

Difficulty: Basic **Category:** Transport Layer (TCP/UDP)

Step 1 (SYN): the client sends a segment with the **SYN** flag set and an initial sequence number, proposing to open a connection. **Step 2 (SYN-ACK):** the server responds with **SYN+ACK** — acknowledging the client's sequence number *and* proposing its own initial sequence number. **Step 3 (ACK):** the client acknowledges the server's sequence number. After this, both sides have confirmed they can send *and* receive, and sequence numbers are synchronized for reliable, ordered delivery.

Common Misconceptions / Traps

Describing it as just "SYN, SYN-ACK, ACK" without being able to say *why* three steps are needed (two-way isn't enough because the initiator never confirms the responder's sequence number was received) is a common shallow-memorization tell.

Follow-Up Questions & What's Being Tested

Follow-up: "What closes a TCP connection, and why does it take four steps (FIN, ACK, FIN, ACK) instead of three?" Tests whether you know closing is asymmetric — each side must independently signal it has no more data, since TCP connections are full-duplex and one side may still have data to send after the other is done.

Q10. What is TCP congestion control, and what does "slow start" actually do?

Difficulty: Intermediate **Category:** Transport Layer (TCP/UDP)

TCP has no way to know the network's true capacity in advance, so it **probes** for it and backs off on loss. **Slow start** begins with a small **congestion window** (cwnd) — historically a few segments — and roughly doubles it each round-trip as ACKs confirm delivery, growing exponentially until either a loss is detected or a threshold (**ssthresh**) is reached, at which point the algorithm switches to the slower, linear growth of **congestion avoidance**. On packet loss, classic TCP (Reno/CUBIC) cuts the window significantly and re-probes more cautiously. Modern algorithms like **CUBIC** (RFC 9438, the Linux default for years) and **BBR** (Google, models the actual bottleneck bandwidth and round-trip time directly instead of purely reacting to loss) refine this reaction to be less punishing on today's high-bandwidth, sometimes lossy-but-not-congested links (e.g., WiFi).

Common Misconceptions / Traps

Calling congestion control and **flow control** the same thing. Flow control protects the *receiver* from being overwhelmed (based on its advertised receive window); congestion control protects the *network* from being overwhelmed (based on inferred/measured capacity). Both can independently limit how much data is in flight.

Follow-Up Questions & What's Being Tested

Follow-up: "Why might BBR outperform CUBIC on a lossy WiFi link?" Tests whether you know CUBIC treats any packet loss as a congestion signal and backs off, while BBR distinguishes real congestion from random loss by modeling bandwidth/RTT directly — valuable on links where loss isn't always congestion.

Q11. A client reports intermittent multi-second stalls on an otherwise-healthy TCP connection. What TCP-level mechanism is the likely suspect, and how would you confirm it?

Difficulty: Advanced **Category:** Transport Layer (TCP/UDP)

The likely suspect is **head-of-line blocking combined with a retransmission timeout**: TCP guarantees *ordered* delivery to the application, so if one segment is lost, every segment received *after* it sits buffered and undelivered to the app until the lost segment is retransmitted and arrives — even though later data already arrived intact. If that loss isn't caught quickly by **fast retransmit** (triggered by duplicate ACKs), TCP falls back to a coarser **retransmission timeout (RTO)**, which can be a second or more, producing exactly this symptom. Confirm with a packet capture (tcpdump, Section 14) looking for duplicate ACKs followed by a retransmission after a visible gap, or with ss -i to inspect the socket's retransmission counters and RTT estimates directly.

Common Misconceptions / Traps

Jumping straight to "it's the network being slow" without distinguishing **latency** (consistently slow) from **stalls** (fine, then a multi-second gap, then fine) — these have different root causes and the fix for one won't address the other.

Follow-Up Questions & What's Being Tested

Follow-up: "What's the difference between fast retransmit and RTO-based retransmit, and why is the gap between them significant?" Tests depth on TCP loss recovery — fast retransmit (three duplicate ACKs trigger immediate resend) is typically much faster than waiting for RTO, so a stall pattern that looks like "occasional single-second-plus freezes" specifically implicates RTO-based recovery, not fast retransmit.

Q12. What is TCP's Nagle's algorithm, and why does it sometimes interact badly with an application sending small, latency-sensitive messages?

Difficulty: Advanced **Category:** Transport Layer (TCP/UDP)

Nagle's algorithm delays sending small TCP segments, buffering them briefly to coalesce with more data before transmitting, in order to avoid flooding the network with many tiny packets (each carrying disproportionate header overhead). It interacts badly with latency-sensitive request/response protocols that send small messages and expect an immediate reply, because Nagle's buffering delay can add up to tens of milliseconds of avoidable latency per message — particularly painful when combined with **delayed ACKs** on the receiving side (the receiver also waits briefly before acknowledging, hoping to piggyback the ACK on a response), which can compound into a much larger stall than either mechanism alone. The standard fix is setting `TCP_NODELAY` on the socket to disable Nagle's algorithm for latency-sensitive traffic.

Common Misconceptions / Traps

Not knowing this is configurable, or assuming it's always beneficial to disable it. Disabling Nagle's algorithm trades a small amount of network efficiency (more, smaller packets) for lower per-message latency — the right call for a chat/gaming/RPC protocol, the wrong call for bulk data transfer where throughput matters more.

Follow-Up Questions & What's Being Tested

Follow-up: "What is the Nagle + delayed-ACK interaction specifically called, and how would you diagnose it in production?" Tests whether you've actually hit this in the wild — it's a classic, named pathology, and the diagnosis is a packet capture showing a consistent ~40ms (a common delayed-ACK timer value) gap between a small write and its ACK.

6 DNS

Q13. Walk through DNS resolution: what happens between typing a hostname and getting back an IP address?

Difficulty: Basic **Category:** DNS

The resolving chain, checked in order until an answer is found: **browser cache** → **OS resolver cache** → the configured **recursive resolver** (often the ISP's, a router's, or a public one like [redacted]). If the recursive resolver has no cached answer, it walks the DNS hierarchy itself: query a **root server** (returns a referral to the correct **TLD server**, e.g. for [redacted]), query that TLD server (returns a referral to the domain's **authoritative name server**), then query the authoritative server directly for the actual record. The answer is returned to the client and cached at each level for its **TTL** (time-to-live). DNS primarily uses **UDP port 53** for queries (falling back to TCP for large responses — historically anything over 512 bytes, or for zone transfers).

Common Misconceptions / Traps

Saying DNS "always uses UDP." It falls back to TCP when a response would be truncated (the truncated/TC bit) or for zone transfers between name servers — an absolute "DNS is UDP" answer misses this.

Follow-Up Questions & What's Being Tested

Follow-up: "What does [redacted] actually do, and when would you reach for it?" Tests hands-on tool familiarity (Section 14) — it manually walks the root → TLD → authoritative chain shown above, which is exactly what you'd use to find *where* a resolution chain is broken or returning stale data.

Q14. Name the common DNS record types and what each is for.

Difficulty: Intermediate **Category:** DNS

A: hostname → IPv4 address. **AAAA**: hostname → IPv6 address. **CNAME**: an alias pointing one hostname to another hostname (cannot coexist with other records on the same name). **MX**: mail server(s) for a domain, with priority values. **TXT**: arbitrary text, commonly used for domain ownership verification and **SPF/DKIM/DMARC** email-authentication policies. **NS**: which name servers are authoritative for a domain. **SOA**: zone metadata (primary name server, admin contact, serial number, refresh/retry/expire timers, and the negative-caching TTL). **PTR**: reverse lookup, IP → hostname, used for reverse DNS.

Common Misconceptions / Traps

Not knowing that a **CNAME** record cannot coexist with other record types at the same name (e.g., you can't have both a CNAME and an MX record on [redacted]) — this is a real, frequently-hit operational constraint, not a trivia footnote.

Follow-Up Questions & What's Being Tested

Follow-up: "Why can't you put a CNAME on your apex/root domain ([redacted] itself)?" Tests understanding that the apex must also hold NS and SOA records, and CNAME's "cannot coexist" rule forbids that — which is exactly why services offer **ALIAS/ANAME** records or flattened-CNAME workarounds for apex domains.

Q15. How does DNS TTL affect both performance and how quickly a change (e.g., a failover) actually takes effect?

Difficulty: Intermediate **Category:** DNS

TTL (time-to-live) tells every resolver and client along the chain how long it may cache a record before re-querying. A high TTL (e.g., 24 hours) means fewer lookups and lower latency for most clients (answered from cache), but it also means that if you change the record — pointing a domain at a new IP during a failover, for instance — clients holding a cached copy won't see the change until their cached TTL expires, regardless of what the authoritative server now says. This is why teams doing planned failovers **lower the TTL in advance** (well before the TTL's own expiry, so the low-TTL record itself has time to propagate) so that, when the actual cutover happens, stale caches expire quickly.

Common Misconceptions / Traps

Assuming a DNS change takes effect "immediately" everywhere once you update the authoritative record. It only takes effect immediately for resolvers with no cached copy; every resolver already holding the old answer keeps serving it until that cached TTL runs out, which is exactly why DNS propagation isn't instantaneous.

Follow-Up Questions & What's Being Tested

Follow-up: "What's DNS cache poisoning, and how does TTL play into the risk?" Tests security awareness — an attacker who successfully injects a forged answer benefits from a *longer* TTL, since the poisoned entry persists in caches longer before a legitimate re-query can correct it.

Q16. Your service is returning connection refused/timeouts for some users but not others, and `dig` from your own machine returns the correct IP. What's your diagnostic approach?

Difficulty: Advanced **Category:** DNS

This pattern — correct resolution from your vantage point, failures for a subset of real users — points away from "DNS is broken" (globally) and toward either (a) **partial/regional propagation**: a recent record change hasn't reached every recursive resolver yet, so some users' resolvers still serve a stale, now-wrong answer; or (b) the DNS answer is *correct* everywhere but a subset of users are being routed to an unhealthy backend behind that IP (a load-balancer or GeoDNS problem, not DNS resolution itself). Diagnose by asking affected users which resolver/ISP/region they're on, querying several public resolvers directly (`dig @8.8.8.8`, `dig @1.1.1.1`, and a resolver local to the affected region if possible) to see whether the *answer itself* differs, and separately checking whether the returned IP is actually healthy from the affected region (a curl/traceroute from a VM in that region, or a synthetic-monitoring service with points of presence there).

Common Misconceptions / Traps

Treating "`dig` works for me" as proof DNS isn't the problem. DNS answers are frequently **not globally uniform** by design (GeoDNS, split-horizon DNS, gradual propagation, per-resolver caching state) — your own successful lookup only proves resolution worked from your specific vantage point, at that moment, through your specific resolver.

Follow-Up Questions & What's Being Tested

Follow-up: "What's split-horizon DNS, and why might it explain an internal-only failure?" Tests whether you know some setups deliberately return different answers for internal vs. external queries, which is a legitimate design (e.g., routing internal traffic directly rather than out and back through a public load balancer) that can also become a footgun during misconfiguration.

7 HTTP/HTTPS Evolution

Q17. What's the actual difference between HTTP and HTTPS?

Difficulty: Basic **Category:** HTTP/HTTPS Evolution

HTTPS is HTTP layered on top of **TLS** (Transport Layer Security, Section 8) — same request/response semantics (methods, headers, status codes), but the TLS layer encrypts the connection, authenticates the server's identity via its certificate, and protects integrity (tampering in transit is detectable). Plain HTTP sends everything — including headers, cookies, and body — in cleartext, readable and modifiable by anything on the network path.

Common Misconceptions / Traps

Answering only "HTTPS is encrypted, HTTP isn't" without mentioning **authentication** (proving you're actually talking to the real server, not an attacker) — encryption alone, without authentication, doesn't stop a MITM attacker from simply presenting their own key and decrypting/re-encrypting traffic transparently (Q23).

Follow-Up Questions & What's Being Tested

Follow-up: "What port does each use by default, and is that port number meaningful or just convention?" Tests basic fluency (80 vs. 443) plus whether you know it's pure convention enforced by neither protocol — either can run on any port.

Q18. What problem does HTTP/2 solve that HTTP/1.1 had, and how?

Difficulty: Intermediate **Category:** HTTP/HTTPS Evolution

HTTP/1.1 sends requests over a connection **sequentially** by default (a response must substantially complete before the next request on that connection is sent, though pipelining existed and was rarely used) — browsers worked around this by opening several parallel TCP connections per host (typically 6), which has its own overhead (each connection pays its own TCP + TLS handshake cost). **HTTP/2** introduces **multiplexing**: many requests and responses share a *single* TCP connection, interleaved as independent **streams**, so a slow response no longer blocks unrelated requests behind it on that connection. It also adds **HPACK** header compression (headers repeat a lot across requests to the same host, so compressing them saves real bandwidth) and **server push** (mostly deprecated in practice now).

Common Misconceptions / Traps

Claiming HTTP/2 "eliminates head-of-line blocking" without qualification. It eliminates *application-layer* (HTTP-level) head-of-line blocking between unrelated requests, but because all streams still share one **TCP** connection, a single lost TCP segment still blocks *all* streams until it's retransmitted — **transport-layer** head-of-line blocking remains, which is precisely the gap HTTP/3 closes (Q19).

Follow-Up Questions & What's Being Tested

Follow-up: "Does HTTP/2 require TLS?" Tests a real-world nuance: the spec doesn't mandate it, but every major browser only implements HTTP/2 over TLS in practice, making it de facto mandatory on the open web.

Q19. Why does HTTP/3 use QUIC/UDP instead of TCP, specifically?

Difficulty: Advanced **Category:** HTTP/HTTPS Evolution

HTTP/3 runs over **QUIC**, which is built on **UDP** rather than TCP, specifically to fix the transport-layer head-of-line blocking that HTTP/2 still has (Q18): because QUIC implements its own per-stream reliability *above* UDP (in user space, not the kernel's TCP stack), a lost packet on one stream only blocks *that stream* — other streams on the same QUIC connection keep delivering data uninterrupted, which is impossible when everything is multiplexed inside one TCP byte-stream. QUIC also integrates **TLS 1.3** into its own handshake (rather than TLS running as a separate layer on top), enabling connection setup in fewer round trips (as low as **0-RTT** for a resumed connection), and supports **connection migration** — a client switching networks (WiFi to cellular) can keep the same QUIC connection alive using a connection ID instead of the traditional 4-tuple (source/dest IP+port), which breaks on a network change under TCP.

Common Misconceptions / Traps

Assuming UDP was chosen because it's "faster" in some vague sense. UDP itself isn't inherently faster than TCP for a single request — QUIC's real advantage is architectural (per-stream loss recovery, integrated 0-RTT TLS, connection migration), not a property of UDP packets being quicker to send.

Follow-Up Questions & What's Being Tested

Follow-up: "What's a practical downside of QUIC/UDP that ops teams actually hit?" Tests production awareness — some corporate/legacy firewalls and middleboxes heavily rate-limit or block UDP traffic (historically associated with less "legitimate" traffic than TCP), and QUIC's encrypted-by-default packet headers make traditional deep packet inspection harder, both of which have caused real deployment friction.

Q20. What is HSTS, and what specific attack does it prevent that a normal HTTP → HTTPS redirect doesn't?

Difficulty: Advanced **Category:** HTTP/HTTPS Evolution

HSTS (HTTP Strict Transport Security) is a response header (`Strict-Transport-Security: max-age=...`) telling the browser "for the next `max-age` seconds, never attempt a plain-HTTP connection to this host again — rewrite every request to HTTPS internally, before any network request is sent." This closes a gap a redirect-based approach leaves open: if a user's *first* request to a site is plain HTTP (e.g., they typed the domain with no scheme, or clicked an old `http://` link), that very first request travels over the network in cleartext before the server ever gets a chance to respond with a redirect — and a MITM attacker on that network can intercept it (an attack class known as **SSL stripping**). Browsers additionally maintain a **preload list** of domains that get this protection built-in, with no need for even one prior visit.

Common Misconceptions / Traps

Believing an HTTP → HTTPS redirect alone is sufficient protection. The redirect *itself* is a plain-HTTP response that can be intercepted or rewritten by an attacker before it ever reaches the browser as a redirect — HSTS's entire value is skipping that vulnerable first plain-HTTP round trip.

Follow-Up Questions & What's Being Tested

Follow-up: "Why can't HSTS protect a user's genuinely first-ever visit to a site (without preloading)?" Tests precise understanding of HSTS's trust-on-first-use model — the header itself must be received once, over a connection, before the browser starts enforcing it, so that very first connection is unprotected unless the domain is on the browser's preload list.

8 TLS & Network Security

Q21. Walk through a TLS 1.3 handshake at a conceptual level.

Difficulty: Basic **Category:** TLS & Network Security

ClientHello: the client sends supported TLS versions, cipher suites, and a random value, plus (in 1.3) its key-share for the key exchange up front. **ServerHello**: the server picks a cipher suite, sends its own key-share and random value, and — critically — its **certificate** (an X.509 certificate, signed by a **Certificate Authority**, binding the server's public key to its domain name). **Certificate verification**: the client validates the certificate's signature chain up to a **root CA** it already trusts (shipped in the OS/browser's trust store), checks the domain name matches, and checks it hasn't expired or been revoked. **Key derivation**: both sides independently compute the same shared session keys from the exchanged key-shares (Diffie-Hellman-based in TLS 1.3) — the actual key material is never transmitted, only the public values needed to derive it. From this point, all traffic is encrypted with the derived symmetric session key. TLS 1.3 does this in **one round trip** (down from two in TLS 1.2), and supports **0-RTT resumption** for a previously-visited server.

Common Misconceptions / Traps

Saying TLS "encrypts everything using the server's certificate." The certificate is used to *authenticate* the server (and carry its public key for the initial key exchange) — actual traffic is encrypted with a symmetric **session key** derived during the handshake, not with the certificate's key directly (asymmetric crypto is far too slow for bulk data).

Follow-Up Questions & What's Being Tested

Follow-up: "What changed between TLS 1.2 and 1.3 that reduced the handshake to one round trip?" Tests currency — TLS 1.3 removed weak/legacy cipher options entirely (simplifying negotiation) and moved the key-share into the ClientHello itself instead of negotiating it in a separate round trip.

Q22. Exactly how does the certificate chain prevent a man-in-the-middle attack?

Difficulty: Intermediate **Category:** TLS & Network Security

An attacker positioned on the network path (a MITM) can intercept and even fully relay traffic, but to impersonate the real server convincingly, they'd need to present a certificate for that domain that the client trusts. Every trusted certificate is signed by a **Certificate Authority** the client's OS/browser already has in its **trust store** (a **root CA**), typically via one or more **intermediate CAs** forming a chain. An attacker cannot forge a valid signature for someone else's domain without either (a) compromising a trusted CA's private signing key, (b) tricking a CA into mis-issuing a certificate for a domain they don't control, or (c) getting their own untrusted/self-signed certificate accepted — which triggers a loud browser warning precisely because it *isn't* chained to a trusted root. This is why **encryption without authentication protects against nothing**: an attacker could still negotiate an encrypted channel using their *own* certificate, but the client would refuse to trust it (or warn loudly) rather than silently connecting.

Common Misconceptions / Traps

Thinking encryption alone ("the traffic is scrambled") is what stops MITM attacks. It's the **authentication** step — verifying the certificate's chain of trust — that actually prevents impersonation; an attacker can trivially set up *their own* encrypted channel if the client doesn't verify who it's actually encrypting to.

Follow-Up Questions & What's Being Tested

Follow-up: "What is certificate pinning, and what real problem does it solve beyond normal chain validation?" Tests depth on defense-in-depth — pinning hard-codes which specific certificate/public key/CA a client will accept for a given host, protecting against a *legitimately* mis-issued or CA-compromised certificate that would otherwise pass normal chain validation.

Q23. A user reports a browser certificate warning on your site. Walk through your diagnostic approach.

Difficulty: Advanced **Category:** TLS & Network Security

First, get the exact warning text/type — it changes the diagnosis entirely. **Expired certificate**: check the cert's validity dates directly (`openssl s_client -connect host:443` openssl x509 -noout -dates|, or `curl -v` which prints certificate details) — likely an automated renewal (e.g., Let's Encrypt/ACME) silently failed. **Hostname mismatch**: the certificate's **Subject Alternative Names** don't include the hostname the user actually requested — check for a recent DNS change pointing a new hostname at an old load balancer/server with an outdated cert, or a missing SAN entry on a multi-domain cert. **Untrusted issuer / self-signed**: either a genuinely misconfigured server, or — worth explicitly considering — a MITM (corporate TLS-inspecting proxy, public WiFi captive portal, or genuinely malicious actor), especially if the report is from one specific user/network and not reproducible elsewhere. **Chain incomplete**: the server is serving only its leaf certificate without the required **intermediate certificate(s)** — some clients/OSes cache intermediates from other sites and mask this, so it may work on your machine and fail for the reporting user; `openssl s_client -connect host:443 -showcerts` shows exactly what chain the server actually sends.

Common Misconceptions / Traps

Assuming "it works fine when I check it" rules out a certificate problem. An incomplete chain is the classic case where it *does* work on your machine (which already trusts/caches the missing intermediate from elsewhere) while failing for the affected user — always verify with a tool that shows exactly what the server sends, not what your browser already trusts.

Follow-Up Questions & What's Being Tested

Follow-up: "How would you tell a legitimate self-signed/expired cert issue apart from an actual MITM in progress?" Tests threat-modeling instinct — scope (one user/network vs. everyone), timing correlation with a known deploy/renewal, and comparing the certificate's actual fingerprint/serial number against what you expect to be serving are the concrete differentiators, not a gut feeling.

Q24. What is TLS session resumption, and why does it matter for performance at scale?

Difficulty: Advanced **Category:** TLS & Network Security

A full TLS handshake (Q21) costs a round trip and non-trivial CPU (asymmetric crypto for the key exchange and certificate verification). **Session resumption** lets a client and server that have already completed a full handshake skip most of that work on a subsequent connection within a validity window. TLS 1.3 does this via **session tickets** (an encrypted blob the server gives the client after the first handshake, containing enough state for the server to resume without server-side storage) enabling **0-RTT** data on the very first flight of the resumed connection — the client can send encrypted application data immediately, before even completing the resumed handshake. At scale (a CDN or load balancer terminating millions of connections), this materially reduces both latency (fewer round trips) and CPU load (skipping expensive asymmetric operations) for the very common case of a client reconnecting to a site it recently visited.

Common Misconceptions / Traps

Not knowing 0-RTT data has a real security trade-off: it's vulnerable to **replay attacks** (an attacker can capture and resend that first 0-RTT flight), so it's only considered safe for **idempotent** requests — production systems generally restrict what can be sent as 0-RTT data for exactly this reason.

Follow-Up Questions & What's Being Tested

Follow-up: "How does session resumption interact with load balancing across many backend servers?" Tests production nuance — if session tickets are encrypted with a key only one server instance knows, resumption fails whenever the client lands on a *different* backend, which is why shared/synchronized ticket keys (or centralizing TLS termination at the load balancer) matter operationally.

9 NAT & Firewalls

Q25. What does NAT actually do, and why does a home router need it?

Difficulty: Basic **Category:** NAT & Firewalls

NAT (Network Address Translation) rewrites the source (and/or destination) IP address — and typically the port — of packets as they cross a network boundary, most commonly translating many private, non-routable addresses (RFC 1918, Q6) to one public IP. The most common form, **PAT** (Port Address Translation, often loosely just called "NAT"), lets many internal devices share one public IP by giving each internal connection a distinct external port, and maintains a translation table mapping (internal IP, internal port) ↔ (public IP, external port) so return traffic gets routed back to the correct internal device. A home router needs this because ISPs typically assign only one public IPv4 address per connection, while a household has many devices needing internet access.

Common Misconceptions / Traps

Describing NAT as a security feature by design. It has a security *side effect* (internal devices aren't directly addressable from the internet, which incidentally blocks unsolicited inbound connections), but that's a byproduct of address translation, not NAT's purpose — a stateful firewall is the actual, intentional security control (Q26).

Follow-Up Questions & What's Being Tested

Follow-up: "Why can two internal devices behind the same NAT both keep separate connections open to the same external server on the same port?" Tests understanding of the full translation tuple — NAT distinguishes connections by the *combination* of internal IP+port (mapped to distinct external ports), not just the destination, so this works fine as long as the NAT table has capacity.

Q26. Stateful vs. stateless firewalls — what's the practical difference in what each can enforce?

Difficulty: Intermediate **Category:** NAT & Firewalls

A **stateless firewall** evaluates each packet independently against a fixed rule set (source/dest IP, port, protocol) with no memory of prior packets — it can't distinguish "a reply to a connection I allowed out" from "an unsolicited inbound packet that happens to have a matching port," so rules must be written defensively and are harder to get both correct and tight. A **stateful firewall** tracks active connections in a **state/connection table** (source/dest IP+port, protocol, TCP state) and can therefore allow *return traffic* for a connection that was legitimately initiated from inside, while still blocking genuinely unsolicited inbound traffic — even on the exact same port a legitimate reply would use. This is what lets a simple "allow outbound, default-deny inbound" policy actually work without also allowing spoofed inbound traffic that happens to match.

Common Misconceptions / Traps

Assuming stateless firewalls are strictly worse and therefore obsolete. Stateless (ACL-style) filtering is still used at high-throughput points (some router ACLs, some cloud network ACLs) precisely because it doesn't need per-connection state, which is cheaper and simpler at very large scale — it's a genuine trade-off, not an outdated technique.

Follow-Up Questions & What's Being Tested

Follow-up: "What happens to a stateful firewall's connection table during failover to a standby unit?" Tests operational depth — high-availability stateful firewall pairs typically synchronize the state table (NAT translations, TCP connection states) to the standby unit so existing connections survive a failover without being dropped and needing to re-establish.

Q27. Two services behind NAT both need to initiate long-lived outbound connections to the same third-party API. One works fine; the other keeps getting silently dropped after a period of inactivity. Why, and what's the fix?

Difficulty: Advanced **Category:** NAT & Firewalls

The likely cause is **NAT/firewall connection-table timeout**: both stateful NAT and stateful firewalls evict idle connection entries from their table after a timeout to bound table size, and if the "broken" service's connection sits idle (no traffic) longer than that timeout while the "working" one sends periodic traffic, only the idle one gets silently evicted — the far end will send a reply to a mapping that no longer exists, and it simply vanishes with no error on either side, since it's the NAT device silently dropping the return packet, not either endpoint refusing the connection. The standard fix is application-level **keepalives**: send small, periodic traffic on the connection (`TCP_KEEPALIVE`, or an application-level heartbeat/ping) at an interval shorter than the NAT/firewall's idle timeout, which resets the connection's activity timer and keeps its table entry alive.

Common Misconceptions / Traps

Debugging this by staring at the application's own logs/error handling. The failure is invisible to the application until it eventually notices the connection is dead (often via its own much longer TCP timeout) — the actual evidence lives in a packet capture or the NAT/firewall device's own connection-table logs, showing the mapping simply disappeared with no RST or FIN from either real endpoint.

Follow-Up Questions & What's Being Tested

Follow-up: "Why might TCP keepalive alone not be enough, and when would you need an application-level heartbeat instead?" Tests real operational experience — OS-default TCP keepalive intervals are often very long (hours, by default on many systems) unless explicitly tuned, so many teams implement a faster application-level heartbeat instead of relying on (or in addition to) tuning OS-level keepalive settings.

10 Load Balancing & Reverse Proxies

Q28. What's the difference between a load balancer and a reverse proxy?

Difficulty: Basic **Category:** Load Balancing & Reverse Proxies

A **reverse proxy** sits between clients and one or more backend servers, forwarding requests on the backend's behalf — a reverse proxy in front of a *single* server (for TLS termination, caching, or hiding the backend's real address) is still a reverse proxy even though it isn't distributing load across anything. A **load balancer** is a reverse proxy that specifically distributes traffic across *multiple* backend instances, using some algorithm (round robin, least connections, consistent hashing) and typically **health checks** to route only to healthy backends. In practice the terms overlap heavily and the same software (nginx, HAProxy, Envoy) commonly does both jobs at once.

Common Misconceptions / Traps

Treating "load balancer" and "reverse proxy" as mutually exclusive categories. Every load balancer performing HTTP-level routing *is* a reverse proxy; the useful distinction is what *additional* thing it's doing (distributing across many backends, with health awareness) rather than a hard boundary between the two terms.

Follow-Up Questions & What's Being Tested

Follow-up: "Where does an API gateway fit relative to these two?" Tests whether you can place a third, closely related term — an API gateway is typically a reverse proxy/load balancer *plus* additional concerns like authentication, rate limiting, and request transformation/routing by API contract rather than just by host/path.

Q29. L4 vs. L7 load balancing — what can each actually see and decide on, and when would you choose one over the other?

Difficulty: Intermediate **Category:** Load Balancing & Reverse Proxies

An **L4** (transport-layer) load balancer makes its routing decision from TCP/UDP-level information only — source/dest IP and port — without parsing the application payload at all; it typically just forwards packets/connections to a backend and gets out of the way, which makes it fast and able to handle very high connection/packet rates with low overhead. An **L7** (application-layer) load balancer terminates and actually parses the request (HTTP, gRPC, etc.), so it can route based on URL path, headers, cookies, or request content, and commonly also handles **TLS termination**, request/response transformation, and detailed observability — at the cost of more CPU per request and an extra hop of latency for that parsing. Choose L4 for raw throughput on non-HTTP or extremely high-volume traffic (e.g., a gRPC service doing huge numbers of short-lived internal RPCs) where content-aware routing isn't needed; choose L7 when routing decisions or request-level behavior (rate limiting, A/B routing by header, canary releases by path) depend on the actual request content.

Common Misconceptions / Traps

Defaulting to "always use L7, it's more capable." A common, real interview trap: for very high-throughput internal service-to-service traffic (e.g., 100K+ req/s gRPC), routing everything through an L7 proxy adds real, avoidable overhead when L4 (or client-side load balancing entirely) would do the job with less latency and CPU cost.

Follow-Up Questions & What's Being Tested

Follow-up: "What is SSL/TLS termination, and why does it usually happen at the load balancer rather than each backend?" Tests whether you understand the trade-off — centralizing TLS termination at the LB simplifies certificate management (one place to renew/rotate) and offloads the CPU cost of the TLS handshake from every backend instance, at the cost of the LB-to-backend hop typically running unencrypted unless a second TLS layer is deliberately added.

Q30. Round robin sends roughly equal traffic to every backend, but one backend is consistently slower under load than the others. What load-balancing algorithm change would you make, and why?

Difficulty: Advanced **Category:** Load Balancing & Reverse Proxies

Round robin distributes *requests* evenly but has no awareness of each backend's actual current load or response time — it will keep sending an equal share to a struggling backend, making its problem worse. Switching to **least connections** (route to whichever backend currently has the fewest active connections) or, better, a **latency-aware/weighted response-time** algorithm directly accounts for a backend that's slow to finish requests by naturally routing less *new* traffic to it while it's backed up, since its active-connection count (or observed latency) will be higher than its healthy peers'. For stateful/caching workloads where a given client should consistently reach the same backend, **consistent hashing** is the relevant technique instead — though it doesn't by itself solve the "one backend is slow" problem and is usually combined with health-aware routing on top.

Common Misconceptions / Traps

Reaching for "just add more backends" as the first answer without diagnosing *why* one backend is consistently slower under identical round-robin load — that asymmetry (bad hardware, a noisy neighbor on shared infrastructure, a cold cache, or a genuinely different code/config version) is itself the more important finding, and load-balancer algorithm changes are a mitigation, not a root-cause fix.

Follow-Up Questions & What's Being Tested

Follow-up: "How do health checks interact with this — would a standard health check even catch a 'slow but technically up' backend?" Tests whether you know a simple binary up/down health check ("does [redacted] return 200?") often won't catch degraded-but-responsive backends, which is why production systems increasingly use **outlier detection** based on latency/error-rate percentiles rather than pure liveness checks.

11 Routing & Switching

Q31. What is a VLAN, and what problem does it solve?

Difficulty: Basic **Category:** Routing & Switching

A **VLAN** (Virtual LAN) logically partitions a single physical switched network into multiple isolated broadcast domains, as if they were separate physical networks, without needing separate physical switches for each. Devices on different VLANs cannot communicate directly at Layer 2 even if plugged into the same physical switch — traffic between VLANs must be explicitly routed (by a router or Layer 3 switch), giving a deliberate, auditable point of control. This solves both a **scaling** problem (broadcast traffic on a large flat network degrades performance for everyone, since every device sees every broadcast) and a **security/segmentation** problem (e.g., keeping a guest WiFi network isolated from internal corporate systems on the same physical switching hardware).

Common Misconceptions / Traps

Assuming VLAN separation alone is sufficient security isolation. VLANs are a Layer 2 segmentation tool, not a security boundary by themselves — a misconfigured trunk port, VLAN hopping attack, or simply a router with routes between the VLANs can undermine the isolation; real security segmentation layers firewall rules *on top of* VLAN boundaries.

Follow-Up Questions & What's Being Tested

Follow-up: "What is a trunk port, and how does 802.1Q tagging work?" Tests hands-on switching knowledge — a trunk port carries traffic for multiple VLANs over one physical link by inserting a VLAN ID tag into each Ethernet frame (802.1Q), which the receiving switch reads to know which VLAN's broadcast domain the frame belongs to.

Q32. What is ARP, and why is it needed even though we already have IP addresses?

Difficulty: Intermediate **Category:** Routing & Switching

ARP (Address Resolution Protocol) maps an IP address to the **MAC address** needed to actually deliver a frame on a local Layer 2 network. IP addresses are a Layer 3 (logical, routable) concept, but Ethernet delivery on a local segment happens at Layer 2 using MAC addresses — before a host can send a packet to another host on the same local network (or to its default gateway, for anything off-network), it needs to know *which physical interface* (MAC address) that IP address currently corresponds to. The sending host broadcasts an ARP request ("who has this IP?") to the local network; the owning device replies with its MAC address, which gets cached (the ARP table/cache) to avoid repeating this for every packet.

Common Misconceptions / Traps

Confusing ARP with DNS. DNS resolves a *hostname* to an *IP address* and can involve remote servers across the internet; ARP resolves an *IP address* to a *MAC address* and is strictly local-network-only (it never crosses a router) — they solve superficially similar-sounding but operationally very different problems at different layers.

Follow-Up Questions & What's Being Tested

Follow-up: "What is ARP spoofing, and why is it a realistic attack on a local network?" Tests security awareness — because ARP has no built-in authentication, a malicious device on the same local segment can send forged ARP replies claiming to own another device's (or the gateway's) IP, redirecting traffic through itself (a classic local-network MITM technique).

Q33. What's the practical difference between a link-state routing protocol (like OSPF) and a distance-vector protocol (like older RIP), and why does it matter at internet scale (BGP)?

Difficulty: Advanced **Category:** Routing & Switching

A **distance-vector** protocol (RIP) has each router share only what it directly knows — its own routing table (destination + a distance metric like hop count) — with its immediate neighbors, who then propagate their own updated view outward; no router has a full picture of the topology, only "how far away" and roughly "via whom." A **link-state** protocol (OSPF) has every router flood information about its own direct links to the *entire* routing domain, so every router independently builds a complete map of the topology and computes shortest paths itself (Dijkstra's algorithm) — generally faster to converge after a change and less prone to certain historical distance-vector problems (like the "count-to-infinity" loop issue), at the cost of more memory/CPU to hold the full topology. **BGP** (Border Gateway Protocol, the protocol that actually routes between autonomous systems across the public internet) is technically a distance-vector-family protocol (path-vector, a refinement that tracks the full AS-path to avoid loops rather than just a metric), chosen at internet scale specifically because flooding full link-state topology information globally, the way OSPF does within a single organization's network, would be completely unworkable across hundreds of thousands of independently-operated networks.

Common Misconceptions / Traps

Treating this as pure trivia ("name the two protocol families") rather than understanding *why* the internet's actual backbone protocol (BGP) had to make different trade-offs than an internal enterprise network's protocol (OSPF) would — the scale and trust-domain difference (one organization vs. thousands of independently operated networks) is the actual reason, not just protocol preference.

Follow-Up Questions & What's Being Tested

Follow-up: "What is a BGP route leak or hijack, and why is it hard to prevent structurally?" Tests awareness of a real, recurring internet-scale incident class — because BGP fundamentally trusts announcements from peers (with only partial, inconsistently-deployed cryptographic validation like RPKI in wide use), a misconfigured or malicious network can announce routes for IP space it doesn't own, and traffic can genuinely follow that bad announcement until it's caught and corrected.

12 End-to-End Flow: What Happens When You Type a URL

Q34. Describe, in full technical detail, what happens between typing `https://example.com` into a browser and the page appearing on screen.

Difficulty: Advanced (Synthesis) **Category:** End-to-End Flow

This question deliberately touches almost every section of this guide — interviewers use it as a single, efficient probe of overall depth. A strong answer walks through, in order:

1. **Parsing & HSTS check (Q20):** the browser parses the input as a URL, and checks its preloaded/cached **HSTS** list — if `example.com` is on it, the browser internally forces `https://` even if the user typed (or omitted) a scheme, skipping any plain-HTTP round trip entirely.
2. **DNS resolution (Section 6):** browser cache → OS cache → recursive resolver → (if uncached) the root → TLD → authoritative-server chain, ultimately returning an IP address for `example.com`.
3. **TCP connection (Section 5):** the browser opens a TCP connection to that IP on port 443 — the **three-way handshake** (Q9).
4. **TLS handshake (Section 8):** ClientHello/ServerHello, certificate presentation and chain validation, key exchange, and derivation of the session key (Q21) — in TLS 1.3, in one round trip, potentially 0-RTT if this is a resumed session (Q24). If HTTP/2 or HTTP/3 is in use, **ALPN** (negotiated during the TLS handshake) tells the browser and server which HTTP version to speak next.
5. **HTTP request/response (Section 7):** the browser sends an HTTP request (method, path, headers including `Host` and cookies) over the now-encrypted connection; the server (commonly behind a **load balancer** performing TLS termination and backend routing, Section 10) processes it and returns a response with a status code, headers, and a body.
6. **Rendering:** if the status is in the 200 range, the browser begins parsing the returned HTML, discovering and fetching additional resources (CSS, JS, images — each potentially repeating steps 2–5, though usually reusing the already-open connection and DNS cache for same-origin resources), building the DOM/CSSOM, and painting the page — with JavaScript execution potentially blocking or altering this process along the way.

Common Misconceptions / Traps

Rattling off the steps in a memorized list without being able to go deep on *any single one* when asked to. The whole point of this question is to see where you can zoom in — a candidate who says "then DNS resolves it" and can't explain *how* (Section 6) when pushed has memorized an outline, not understood the system.

Follow-Up Questions & What's Being Tested

Follow-up: "Where, specifically, could this flow fail or be slow, and how would you diagnose each?" Tests whether you can connect this synthesis question directly to the Troubleshooting Playbook (Section 13) — a strong candidate maps each step above to a concrete failure mode (DNS misconfiguration, TCP connect timeout, certificate error, backend 5xx, slow client-side rendering) and the specific tool that would confirm it.

13 Troubleshooting Playbook

Every scenario below follows the same shape, matching how a strong candidate should actually reason under pressure: **symptoms → hypotheses → checks → commands/tools → root cause → fix**. These are drawn from the shape of real Glassdoor/SRE-interview scenario questions and Google's SRE incident-response material, not invented in isolation.

T1 — "The site is down" (the classic opener)

Symptoms: Users report the site is completely unreachable; alerts are firing.

Hypotheses (in likely-first order): recent deploy/config change; DNS pointing at the wrong place; all backends unhealthy behind the load balancer; upstream dependency (database, auth service) down; a certificate expired.

Checks & commands:

```
curl -v https://example.com/health        # does the LB/edge respond at all?
dig example.com                           # is DNS still resolving correctly?
curl -vI https://example.com              # inspect status code + cert details
ss -tan | grep :443                       # is anything even listening locally,
    if on the box?
```

Root cause & fix: *First move regardless of hypothesis:* check what changed most recently (deploy, config push, DNS/cert renewal) — most real outages trace to a recent change, and rolling that back is almost always faster than root-causing forward under pressure. Confirm via the specific check above (DNS answer wrong → revert DNS; [illegible] shows TLS error → certificate issue; [illegible] connects but every response is 5xx → backend/dependency issue, escalate to that service's own playbook).

T2 — Intermittent connection timeouts, not total outage

Symptoms: Some requests succeed; others hang and time out. No clear pattern by user report alone.

Hypotheses: one unhealthy backend behind the load balancer still receiving traffic; a saturated connection pool; intermittent packet loss on one network path; a NAT/firewall idle-timeout issue (Q27) for long-lived connections specifically.

Checks & commands:

```
for i in $(seq 1 20); do curl -o /dev/null -s -w "%{http_code} %{time_total}\n" \
  https://example.com/health; done          # look for a pattern across repeated
    hits
mtr example.com                             # per-hop loss/latency over time,
    better than one-shot traceroute
ss -ti                                      # inspect retransmit counts / RTT on
    active sockets
```

Root cause & fix: If failures correlate with hitting one specific backend (check load-balancer access logs for backend IP per request), pull that backend from rotation and investigate it in isolation. If [illegible] shows consistent loss at one specific hop across many runs, it's a network-path issue — escalate to network/cloud provider with that evidence. If failures cluster on long-idle connections specifically, suspect NAT/firewall timeout (Q27) and add keepalives.

T3 — DNS resolves differently in different regions

Symptoms: The service works from the office/VPN but fails (or resolves to a different, wrong IP) for users in another region.

Hypotheses: intended GeoDNS/split-horizon behavior misfiring for that region; a regional DNS propagation delay after a recent change; a regional recursive resolver serving stale cached data.

Checks & commands:

```
dig example.com @8.8.8.8                    # query a specific public resolver
    directly
dig example.com @1.1.1.1
dig example.com +trace                      # walk the full resolution chain
    yourself
```

Root cause & fix: Compare the actual returned IP across several resolvers/regions (public resolvers, or a synthetic check from a VM in the affected region). If the authoritative answer itself is wrong for that region, fix the GeoDNS/split-horizon configuration. If the authoritative answer is correct everywhere but a specific resolver is stale, the fix is time (TTL expiry) or, for a genuinely broken resolver, working around it on the affected users' side — this is rarely something you can force-fix remotely.

T4 — High latency under load, no errors

Symptoms: Requests all eventually succeed, but p99 latency has climbed sharply under peak traffic; no 5xx errors.

Hypotheses: TCP congestion-window/queueing effects at high connection counts (Q10); a saturated connection pool causing queuing before a request is even sent; a slow downstream dependency whose latency is being fully absorbed rather than surfaced as an error.

Checks & commands:

```
ss -s                                       # summary of socket states -- look
    for large TIME-WAIT/queue counts
curl -w "@curl-format.txt" -o /dev/null -s https://example.com  # break down DNS/
    connect/TLS/TTFB timing
tcpdump -i eth0 host example.com -w capture.pcap   # capture for offline analysis
     of retransmits/timing
```

Root cause & fix: Use the `curl` timing breakdown to isolate *which phase* is slow (DNS, TCP connect, TLS handshake, or time-to-first-byte from the app itself) — these have completely different fixes. A slow TTFB with fast connect/TLS points at the application/backend, not the network; a slow connect phase specifically under load points at connection-pool exhaustion or SYN queue backlog on the server.

T5 — TLS/certificate errors for a subset of clients

Symptoms: Some clients get certificate warnings or TLS handshake failures; others connect fine.

Hypotheses: incomplete certificate chain being masked by cached intermediates on most clients (Q23); a recently-rotated cert not yet propagated to every load-balancer/edge node; genuinely old clients that don't support the negotiated TLS version/cipher suite; a MITM-capable network (corporate proxy, captive portal) for the affected subset specifically.

Checks & commands:

```
openssl s_client -connect example.com:443 -showcerts   # see exactly what chain
    is served
openssl s_client -connect example.com:443 | openssl x509 -noout -dates   #
    validity window
curl -v https://example.com 2>&1 | grep -i "SSL\|TLS\|certificate"
```

Root cause & fix: Confirm the server-sent chain is complete (missing intermediate is the single most common cause of "works for me, fails for them"); if complete and correct, correlate failures by client network/version to distinguish a genuine MITM/proxy environment from a stale edge-node certificate that hasn't finished rolling out.

T6 — A service can't reach a third-party API it used to reach fine

Symptoms: Outbound calls to an external API started failing (timeouts or connection refused) with no code change on your side.

Hypotheses: the third party changed/rotated their IP or certificate; an egress firewall/security-group rule changed; DNS for the third-party domain is resolving incorrectly; the third party is actually down or rate-limiting you.

Checks & commands:

```
dig api.thirdparty.com                          # has their DNS answer changed?
curl -v https://api.thirdparty.com/health       # connect refused vs timeout vs TLS
    error tells a different story
traceroute api.thirdparty.com                   # where does the path actually stop?
ip route get <resolved-ip>                      # confirm YOUR routing/egress path
    for that destination
```

Root cause & fix: Connection refused (fast failure) usually means you *are* reaching a host, but nothing is listening on that port there — consistent with the third party having moved/decommissioned an endpoint. **Timeout** (no response at all) is consistent with either an egress firewall rule silently dropping the traffic on your side, or a routing black hole — traceroute showing the path stopping at your own edge/firewall vs. somewhere further out distinguishes these.

T7 — Load balancer returns 502/504 for some requests

Symptoms: Clients intermittently see **502 Bad Gateway** or **504 Gateway Timeout** from the load balancer, while backend logs show the request was never received, or was received but the response never made it back.

Hypotheses: a backend closed the connection during a request (, backend crashed/restarted mid-request); backend response exceeded the load balancer's timeout ( ); the load balancer's idle-connection reuse to a backend raced against the backend closing that same idle connection (a classic "connection reset while reusing a pooled backend connection" race).

Checks & commands:

```
# on the load balancer / edge:
tail -f /var/log/nginx/error.log | grep -i "upstream\|502\|504"
# compare LB-side request timestamps against backend access logs for the same
    request ID
```

Root cause & fix: 502s clustered around backend deploys/restarts: expected — fix by draining connections gracefully before restart (configure the deploy process to deregister from the load balancer, wait for in-flight requests to finish, *then* restart). **504s correlated with slow endpoints:** either raise the timeout deliberately (if the slow response is legitimate) or fix the underlying slow backend call — raising the timeout without investigating just hides the real problem longer. **Sporadic 502s with no deploy correlation:** suspect the idle-connection-reuse race — align the load balancer's upstream keepalive timeout to be *shorter* than the backend's own idle-connection timeout, so the LB never tries to reuse a connection the backend has already decided to close.

14 Tools Quick Reference

The Troubleshooting Playbook above assumes fluency with these. Know at minimum what each line below actually does, not just that the tool exists.

`curl` — HTTP client, request/response inspection

```
curl -v https://example.com                 # verbose: shows TLS handshake + full
    request/response headers
curl -I https://example.com                 # HEAD request -- headers only, fast
    status/cert check
curl -o /dev/null -s -w "%{http_code} %{time_total}\n" https://example.com
                                             # status code + total time, good for
    scripted repeated checks
curl -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb
    :%{time_starttransfer}\n" \
  -o /dev/null -s https://example.com       # per-phase timing breakdown (T4)
```

`dig` / `nslookup` — DNS lookups

```
dig example.com                             # A record lookup against your
    configured resolver
dig example.com @8.8.8.8                    # query a SPECIFIC resolver directly (T3
    )
dig example.com +trace                      # walk the full root -> TLD ->
    authoritative chain yourself
dig example.com MX                          # query a specific record type
nslookup example.com                        # simpler, cross-platform alternative (
    esp. on Windows)
```

`traceroute` / `mtr` — path discovery

```
traceroute example.com                      # one-shot hop-by-hop path + per-hop
    latency
mtr example.com                             # continuous traceroute + loss % per hop
     over time (T2, T6)
```

`traceroute` works by sending packets with increasing TTL and reading the ICMP Time Exceeded reply each router sends when TTL hits zero (Q3) — it shows the path and per-hop response time, but a one-shot run does **not** reliably measure packet loss; `mtr` repeats this continuously and is the better tool when loss, not just latency, is the question.

`tcpdump` — packet capture

```
tcpdump -i eth0 host example.com            # capture traffic to/from a specific
    host
tcpdump -i eth0 port 443                    # capture traffic on a specific port
tcpdump -i eth0 -w capture.pcap             # write to a file for offline analysis (
    e.g. in Wireshark)
tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0'   # filter to just SYN packets,
    useful for connection-rate issues
```

`ss` — socket statistics (modern replacement for `netstat`)

```
ss -tan                                    # all TCP sockets, numeric (no DNS
    lookups slowing it down)
ss -tan state established                  # only established connections
ss -i                                      # internal TCP info: RTT, congestion
    window, retransmits (Q11)
ss -s                                      # summary: counts of sockets per state (
    T4)
```

`ip` — modern replacement for `ifconfig`/`route`

```
ip addr show                               # interfaces and their assigned
    addresses
ip route show                              # the local routing table
ip route get 8.8.8.8                       # which interface/route would be used to
     reach a destination (T6)
ip neigh show                              # the local ARP/neighbor table (Q32)
```

`openssl s_client` — manual TLS inspection

```
openssl s_client -connect example.com:443 -showcerts       # full certificate chain
    as served (Q23)
openssl s_client -connect example.com:443 </dev/null 2>/dev/null \
  | openssl x509 -noout -dates -subject -issuer            # validity window,
    subject, issuer, no interactive session
```

Final Notes

Two habits separate strong answers from memorized ones throughout every section above: naming the **specific bolded term** an interviewer is listening for (not just the general idea), and being able to say *how you'd actually check it* rather than only what the correct answer is in theory. The Troubleshooting Playbook exists because most real interviews — SRE ones especially — reward the second skill more than the first.